Library Management System

Documentation

Overview

A role-based **Library Management System** written in modern C++ (header-driven implementation) that supports:

- Book management (add, remove, search, display)
- User management (Student, Faculty, Librarian)
- Borrow/return with due dates and fines
- Persistent storage via CSV files
- **Secure authentication** with **SHA-256** password hashing

Project Structure (Files & Responsibilities)

MAIN.CPP Program entry point; constructs `LibrarySystem` and calls `start()` to run the CLI.

LIBRARYSYSTEM.H

Application orchestrator. Initializes data from CSVs, loads credentials, wires `Library`, `Authentication`, and user `Account`s. Hosts the interactive menus for Student/Faculty/Librarian and the common flows (borrow/return, add/remove book, add user, change password).

LIBRARY.H

Core domain container. Owns and manages all `Book*`, `User*`, and `Account*`. Provides fast lookup (`findBook`, `findUser`), add/remove operations, and display helpers. Persists updates through `File_Handler`.

USER.H **User model hierarchy**. Abstract base `User` with interface for limits/policies; concrete roles: `Student`, `Faculty`, `Librarian`. Each role defines max books, borrowing period, and fine rate (if any).

BOOK.H **Book model**. Title, author, publisher, year, ISBN, and status (`Available`/`Borrowed`); simple pretty-printer and accessors.

ACCOUNT.H

Per-user state. Tracks borrowed books with timestamps, computes fines, exposes actions: `borrow(Book*)`, `returnBook(Book*)`, `displayBorrowedBooks()`, `payFines()`, and an accessor to the internal borrowed list (used for persistence on startup).

FILEHANDLER.H

Persistence layer. Reads/writes:

- `books.csv` (book catalog)

- `users.csv` (registered users)
- `borrowed_books.csv` (per-user borrowed items with timestamps)

Handles CSV I/O, basic cleaning/normalization, and rehydrates `borrowed` status on load.

AUTHENTICATION.H

Credential store + login. Loads/saves `credentials.csv` and verifies credentials using SHA-256 (no plaintext). Exposes: `authenticate()`, `getUserRole()`, `addCredentials()`, `changePassword()`, `loadCredentials()`, `saveCredentials()`.

SHA256.H

Header-only SHA-256 implementation (no external libraries). Provides `SHA256::hash(std::string)` used by `Authentication.h`.

Data Persistence (CSV Formats & Behavior)

- `books.csv` — header: `Title,Author,Publisher,Year,ISBN,Status`
  Example: `The Pragmatic Programmer,Andrew Hunt,Addison-Wesley,1999,9780201616224,A`
- `users.csv` — header: `Name,ID,Role`
  Example: `Alice,u1,Student`
- `credentials.csv` — rows: `UserID,PasswordHash,Role` (passwords are **SHA-256** hashes, not plaintext)
- `borrowed_books.csv` — header: `UserID,BookTitle,ISBN,BorrowedTime` (UNIX epoch seconds)

Note: On first run, `books.csv` and `users.csv` *must exist* (seed files). The system will then maintain them. If `borrowed_books.csv` is missing, it starts clean and is created on save.

Build & Run

Windows, Linux, macOS (g++)

```
g++ -std=c++17 main.cpp -o library_system
./library_system    # Windows: .\library_system.exe
```

Authentication & Security (Password Encryption)

- All passwords are stored as **SHA-256 hashes** using `SHA256.h`; no plaintext is written to disk.
- During login, the input password is hashed and compared with the stored hash in `credentials.csv`.
- Users can securely change their password via the menu (`changePassword` rehashes and saves).

- **Default passwords on user creation (via Librarian):**
  - `Student` → `pass123`
  - `Faculty` → `prof123`
  - `Librarian` → `lib123`
- Recommended operational practice: users should change the default on first login.

Roles & Policies

- **Student**: up to 3 books, 15-day period, fine rate: 10.0/day overdue.
- **Faculty**: up to 5 books, 30-day period, fine rate: 0.0.
- **Librarian**: management role (add/remove books and users); not intended to borrow.

How It Works (Flow)

1. **Startup**: `LibrarySystem` loads books/users, constructs `Accounts` per user, restores borrowed status from `borrowed_books.csv`, and loads credential hashes.
2. **Login**: user enters ID & password; `Authentication` hashes the input and validates against `credentials.csv`; role is resolved.
3. **Role Menus**:
   - Student/Faculty: view catalog, borrow by ISBN, return, view borrowed items, pay fines, change password.
   - Librarian: add/remove books, list books/users, add users (assigns default password and immediately persists hashed credentials), change password.
4. **Borrow/Return**: `Account` enforces limits/due dates and adjusts `Book` status; `File_Handler` persists updates and borrows with timestamps.

Data Structures & Efficiency

- `unordered_map<string, Book*> books` — O(1) average ISBN lookup for borrow/return and catalog ops.
- `unordered_map<string, User*> users` — O(1) average user ID lookup for login/session binding.
- `unordered_map<string, Account*> accounts` — O(1) average access to per-user state (borrowed list, fines).
- `vector<...>` — compact, cache-friendly storage for iteration over books/users when saving or listing.
- Minimal allocations and explicit ownership: `Library` destructor deletes managed pointers to prevent leaks.

Notable Behaviors & Edge Cases

- Duplicate ISBNs are rejected on add.
- A borrowed book cannot be removed.
- CSV parsing includes basic normalization (e.g., ISBN trim/cleanup).

Features Summary (What It Does)

- Secure login with hashed passwords; per-role capabilities
- Fast book/user lookup; borrow/return with due dates and fines
- Persistent state across runs via CSVs
- Simple, readable CLI for quick evaluation

Possible Extensions

- Salted hashing and account lockout/backoff policies
- Reservations/notifications, search by fields, and audit logs
- GUI frontend; automated tests (CLI I/O scenarios)